# CNN: Numerical, Conceptual Questions

*MCA / Image Processing and Deep Learning – Practice Set*

Note: In CNN calculations, the standard cross-correlation convention is used: the kernel is applied directly without flipping it.

## Question 1 – Convolution + ReLU [6 Marks]

Given the 5×5 grayscale image and 3×3 filter below, use stride = 1 and no padding. Calculate the complete feature map, apply ReLU, and explain what pattern the filter is likely to detect.

**Input image I**

| | | | | |
|---|---|---|---|---|
| 1 | 2 | 3 | 0 | 1 |
| 2 | 3 | 4 | 1 | 2 |
| 1 | 2 | 0 | 2 | 3 |
| 0 | 1 | 2 | 3 | 1 |
| 1 | 2 | 1 | 0 | 2 |

**Filter K**

| | | |
|---|---|---|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

### Solution – Step 1: Output size

$$(5-3)/1 + 1 = 3 \rightarrow 3\times3 \text{ feature map}$$

### Step 2: Compute each position

For example, the top-left position uses:

$$1(1)+0(2)+(-1)(3)+1(2)+0(3)+(-1)(4)+1(1)+0(2)+(-1)(0) = -3$$

Repeating the same calculation for all nine positions gives:

**Convolution feature map**

| | | |
|---|---|---|
| -3 | 4 | 1 |
| -3 | 0 | 0 |
| -1 | 0 | -3 |

### Step 3: Apply ReLU

$$\text{ReLU}(x)=\max(0,x)$$

**ReLU feature map**

| | | |
|---|---|---|
| 0 | 4 | 1 |
| 0 | 0 | 0 |
| 0 | 0 | 0 |

### Step 4: Interpretation

The filter has negative weights on the left and positive weights on the right. It responds strongly when the right side of a local region is brighter than the left side, so it can detect a vertical intensity transition/edge. The sign of the filter may be reversed without changing the basic edge orientation; the polarity changes.

## Question 2 – Output Size and Trainable Parameters [6 Marks]

An RGB image of size 64×64×3 is given to a convolutional layer with 32 filters of size 3×3, stride 1 and padding 1. Bias is enabled. Calculate the output size, trainable parameters, and total number of output activations. Explain why 32 filters do not mean a 32×32 filter.

### Solution – Step 1: Output dimensions

$$\text{Output size} = ((64-3+2\times1)/1)+1 = 64$$

$$\text{Output} = 64\times64\times32$$

### Step 2: Number of parameters per filter

$$3\times3\times3 = 27 \text{ weights} + 1 \text{ bias} = 28 \text{ parameters}$$

**Step 3: Parameters for 32 filters**

28×32 = 896 trainable parameters

**Step 4: Output activations**

64×64×32 = 131,072 activations

| Layer | Calculation | Output |
|---|---|---|
| Input | Given | 32×32×3 |
| Conv 1 | ((32−3+2)/1)+1 | 32×32×16 |
| Pool 1 | ((32−2)/2)+1 | 16×16×16 |
| Conv 2 | ((16−3+2)/1)+1 | 16×16×32 |
| Pool 2 | ((16−2)/2)+1 | 8×8×32 |

**Step 5: Concept**

A filter spans the complete input depth, so each filter is 3×3×3 for an RGB input. The 32 refers to the number of different filters, and therefore to the 32 output feature maps.

## Question 3 – Max Pooling [5 Marks]

A convolutional layer produces the following 4×4 feature map. Apply 2×2 max pooling with stride 2. Then explain what information is retained and what is lost.

**Feature map F**

| | | | |
|---|---|---|---|
| 1 | 3 | 2 | 4 |
| 5 | 6 | 1 | 2 |
| 2 | 8 | 7 | 3 |
| 4 | 2 | 9 | 1 |

**Solution**

Top-left block: max(1,3,5,6)=6

Top-right block: max(2,4,1,2)=4

Bottom-left block: max(2,8,4,2)=8

Bottom-right block: max(7,3,9,1)=9

**Pooled feature map**

| | |
|---|---|
| 6 | 4 |
| 8 | 9 |

The strongest activation in each 2×2 region is retained. Exact positions and smaller activations inside each block are discarded, so pooling reduces spatial resolution and computational load but can remove fine details.

## Question 4 – Complete CNN Pipeline and Parameter Count [8 Marks]

An input image is 32×32×3. The CNN has: Layer 1: Conv 3×3, 16 filters, stride 1, padding 1; Layer 2: Max Pool 2×2, stride 2; Layer 3: Conv 3×3, 32 filters, stride 1, padding 1; Layer 4: Max Pool 2×2, stride 2. Determine the output size after every layer, the trainable parameters in the two convolution layers, and the final feature-map size before a fully connected layer.

**Solution**

**Parameters in Conv 1**

(3×3×3 + 1)×16 = (27+1)×16 = 448

**Parameters in Conv 2**

(3×3×16 + 1)×32 = (144+1)×32 = 4,640

**Final feature-map size**

8×8×32 = 2,048 values

## Question 5 – Why CNN Instead of a Fully Connected Network? [6 Marks]

## Question 6 – ReLU and Non-linearity [5 Marks]

Given the feature map below, apply ReLU, count the zeros introduced, and explain why a CNN needs a non-linear activation function.

| | | |
|---|---|---|
| -3 | 2 | -1 |
| 4 | -5 | 6 |
| -2 | 1 | -7 |

## Question 7 – HARD: Receptive Field Calculation [8 Marks]

A CNN receives an image and applies: Conv 3×3, stride 1 → MaxPool 2×2, stride 2 → Conv 3×3, stride 1 → MaxPool 2×2, stride 2. Assume no padding. Determine the size of the receptive field of one unit in the final feature map with respect to the original image. Explain why receptive field matters in image recognition.

## Question 8 – HARD: Complete Parameter Budget [8 Marks]

A grayscale CNN uses the following architecture: Input 28×28×1 → Conv 3×3, 8 filters, stride 1, padding 1 → MaxPool 2×2, stride 2 → Conv 3×3, 16 filters, stride 1, padding 1 → MaxPool 2×2, stride 2 → Fully Connected 32 neurons → Output layer 10 neurons. Calculate the output size after each layer and the total number of trainable parameters.

### Solution – Output sizes

| Layer | Output |
|---|---|
| Input | 28×28×1 |
| Conv 1 | 28×28×8 |
| Pool 1 | 14×14×8 |
| Conv 2 | 14×14×16 |
| Pool 2 | 7×7×16 |
| Flatten | 7×7×16 = 784 |
| FC | 32 |
| Output | 10 |

## Question 9 – HARD: Small-Defect Detection in Industrial Inspection [8 Marks]

A factory uses a CNN to detect tiny cracks on a metal component. The cracks are only 2–3 pixels wide. The engineer adds repeated 2×2 max-pooling layers and observes that some cracks disappear from the final feature maps. Explain why this can happen. Would adding even more max-pooling necessarily improve the model? Suggest two architecture/data changes and justify them.

### Solution

Max pooling reduces spatial resolution. A 2×2 pooling window replaces four values with one value, so exact locations of very small structures can be lost. Repeated pooling compounds this loss. A 2–3 pixel crack may be merged with surrounding background or may fail to produce a sufficiently strong pooled activation.

Adding more pooling is therefore not necessarily helpful for this task. It can improve computational efficiency and enlarge the effective context, but it also reduces fine spatial detail.

Possible improvements: (1) reduce the number/frequency of pooling operations or use smaller downsampling; (2) use higher-resolution input crops/patches and targeted data augmentation so the network sees enough examples of tiny cracks. Depending on the project, skip connections or multi-scale feature fusion can also help preserve fine detail.

## Question 10 – HARD: Real-World CNN Failure and Generalization [10 Marks]

A CNN classifies chest X-ray images into two classes. It achieves 99% training accuracy but performs much worse on images from another hospital. The second hospital uses a different scanner and image-processing pipeline. Answer: (a) identify two plausible causes, (b) explain why simply increasing the number of CNN layers is not guaranteed to solve the problem, (c) suggest two data/preprocessing changes, and (d) explain the difference between training accuracy and generalization.

### Solution

(a) Two plausible causes are distribution shift and overfitting. The second hospital may have different contrast, intensity distribution, resolution, noise characteristics, or acquisition conditions. The model may also have learned patterns specific to the training data rather than general disease-related features.

(b) Increasing depth increases model capacity, but greater capacity can also increase overfitting and computational cost. A deeper network does not automatically remove a mismatch between training and deployment data.

(c) Useful changes include consistent or robust preprocessing/normalization across sites, and data augmentation that varies contrast, intensity, noise, scale, and other realistic acquisition conditions. Adding representative data from the second site can also help.

(d) Training accuracy measures performance on the data used to fit the model. Generalization refers to performance on unseen data from the target population or deployment setting. A model can memorize training-specific patterns and still generalize poorly.

## Bonus Question – CNN Interpretation Trap [5 Marks]

A student states: “The first CNN layer detects complete objects such as cars and faces, while the deepest layer detects only simple edges.” Is the statement correct? Explain using hierarchical feature learning.

### Solution

The statement is reversed. Early convolutional layers generally learn simple local patterns such as edges, orientations, and textures. Middle layers combine these into more complex motifs or parts. Deeper layers integrate broader spatial context and can represent higher-level object structures. The exact features learned depend on data, architecture, optimization, and task; the hierarchy is a useful conceptual description rather than a strict rule for every neuron.